计算机系统 I：时序逻辑 (Sequential Logic) 复习讲义

目录

1 时序逻辑基础 (Foundations of Sequential Logic) 2
1.1 时序电路的组成 (Sequential Circuit Model) . . . 2
1.2 同步与异步时序电路 (Synchronous vs. Asynchronous) . . . 2
1.3 门延迟、反馈与状态 (Delay, Feedback and State) . . . 2

2 基本存储元件 (Basic Storage Elements) 3
2.1 S-R 锁存器 (S-R Latches) . . . 3
2.2 带时钟的 S-R 锁存器 (Clocked S-R Latch) . . . 4
2.3 D 锁存器 (D Latch) . . . 4
2.4 锁存器的时序问题 (Latch Timing Problem) . . . 4
2.5 主从触发器 (Master-Slave Flip-Flop) . . . 4
2.6 边沿触发 D 触发器 (Edge-Triggered D Flip-Flop) . . . 5
2.7 直接输入与初始化 (Direct Inputs) . . . 5
2.8 触发器时序参数 (Flip-Flop Timing Parameters) . . . 5
2.9 J-K 与 T 触发器 (J-K and T Flip-Flops) . . . 6

3 同步时序逻辑分析 (Synchronous Sequential Logic Analysis) 6
3.1 分析任务与一般流程 . . . 6
3.2 状态表与状态图 (State Table and State Diagram) . . . 7
3.3 分析例 1：两个 D 触发器电路 . . . 7
3.4 Moore、Mealy 与混合输出 . . . 7
3.5 等价状态 (Equivalent States) . . . 8
3.6 分析例 2：三触发器无输入电路 . . . 8
3.7 系统级时序约束 (Circuit and System Level Timing) . . . 9

4 同步时序逻辑设计 (Synchronous Sequential Logic Design) 9
4.1 设计流程 (Design Procedure) . . . 9
4.2 状态图构造：序列识别器 (Sequence Recognizer) . . . 10

4.3 例：识别序列 1101 的 Mealy 模型 10
4.4 例：识别 1101 的 Moore 模型 10
4.5 状态编码 (State Assignment) 11
4.6 状态编码例：Counting vs. Gray 11
4.7 状态编码规则例 12
4.8 设计例 3：模 3 累加器 13
4.9 未用状态与自启动 (Unused States and Self-Starting) 13

5 经典时序逻辑电路 (Classic Sequential Logic Circuits) 14
5.1 寄存器 (Registers) 14
5.2 使能与时钟门控 (Enable and Clock Gating) 14
5.3 寄存器传送与微操作 (Register Transfers and Microoperations) 14
5.4 寄存器传送结构 (Register Transfer Structures) 15
5.5 移位寄存器 (Shift Registers) 15
5.6 计数器 (Counters) 16
5.7 波纹计数器 (Ripple Counters) 17
5.8 同步计数器 (Synchronous Counters) 17
5.9 模 N 计数器与 BCD 计数器 17
5.10 模 N 计数的实现方法 19

6 全章总结 (Summary) 20

1 时序逻辑基础 (Foundations of Sequential Logic)

1.1 时序电路的组成 (Sequential Circuit Model)

时序电路的核心区别在于：输出不仅取决于**当前输入**，还取决于**过去输入形成的当前状态**。因此，一个典型时序电路由两部分构成：

- **存储元件 (Storage Elements)**：锁存器 (Latches) 或触发器 (Flip-flops)，负责保存状态。
- **组合逻辑 (Combinational Logic)**：根据外部输入和当前状态产生输出与下一状态。

常用数学描述如下：

$$\text{Next State} = f(\text{Inputs}, \text{Present State})$$

$$\text{Output}_{\text{Mealy}} = g(\text{Inputs}, \text{State}), \qquad \text{Output}_{\text{Moore}} = h(\text{Present State})$$

时序电路的三类信号

- **Inputs**：来自外部环境的输入信号。
- **State / Present State**：存储元件当前保存的状态。
- **Next State**：组合逻辑送回存储元件、准备在下一时刻保存的新状态。

1.2 同步与异步时序电路 (Synchronous vs. Asynchronous)

时序电路可按存储元件观察输入、改变状态的时间方式分类：

- **同步时序电路 (Synchronous Sequential Circuits)**：行为只在离散时刻定义，存储元件只在时钟信号相关的时刻观察输入并改变状态。同步抽象让复杂数字系统可设计、可分析。
- **异步时序电路 (Asynchronous Sequential Circuits)**：行为由任意时刻输入变化及变化顺序决定。如果把时钟也看作普通输入，严格来说所有电路都可视为异步；但工程设计中通常采用同步抽象。

1.3 门延迟、反馈与状态 (Delay, Feedback and State)

课件用一个 2 输入多路选择器说明了延迟的重要性。理想功能为：

$$Y = \begin{cases} A, & S = 0 \\ B, & S = 1 \end{cases}$$

由于反相器和门电路存在传播延迟，选择信号切换时可能出现短暂毛刺 (Glitch)。如果把输出 Y 反馈到输入 A，电路在 $S = 0$ 时会变成：

$$Y(t) \text{ depends on } Y(t - \Delta)$$

此时 Y 不再只是组合逻辑输出，而是保存了过去的值，成为**状态**。

- 当 $S=1$ 时，$Y=B$，输出跟随输入。
- 当 $S=0$ 时，Y 维持先前值，表现出“记忆”。
- 若反馈路径中再加入反相器，$S=0$ 时电路会不断翻转，形成不稳定振荡器，可粗略产生时钟，但不可作为严谨同步设计的基础。

2 基本存储元件 (Basic Storage Elements)

2.1 S-R 锁存器 (S-R Latches)

S-R 锁存器通过两个门的交叉耦合实现，是最基本的双稳态存储结构。不同实现方式的输入有效电平不同。

NAND S-R Latch

NAND 型 S-R 锁存器通常是**低有效输入**：

S	R	Q^+	$\overline{Q}^+$	**含义**
1	1	Q	$\overline{Q}$	保持原状态
0	1	1	0	Set，置 1
1	0	0	1	Reset，清 0
0	0	1	1	禁止输入，释放后可能不稳定

NOR S-R Latch

NOR 型 S-R 锁存器通常是**高有效输入**：

S	R	Q^+	$\overline{Q}^+$	**含义**
0	0	Q	$\overline{Q}$	保持原状态
1	0	1	0	Set，置 1
0	1	0	1	Reset，清 0
1	1	0	0	禁止输入，释放后可能不稳定

易错点：NAND 与 NOR 的禁止输入不同

- NAND S-R Latch：$S=R=0$ 禁止。
- NOR S-R Latch：$S=R=1$ 禁止。
- 禁止输入会破坏 Q 与 $\overline{Q}$ 互补关系，释放后状态可能不可预测。

2.2 带时钟的 S-R 锁存器 (Clocked S-R Latch)

在基本 NAND S-R 锁存器前加入两个 NAND 门，可得到带控制端 C 的 S-R 锁存器。只有当 $C = 1$ 时，S 和 R 才被观察；当 $C = 0$ 时保持状态。

C	S	R	$Q(t+1)$
0	X	X	No change
1	0	0	No change
1	1	0	1, Set
1	0	1	0, Clear
1	1	1	Undefined

2.3 D 锁存器 (D Latch)

D 锁存器在 S-R 锁存器基础上增加反相器，使 $S = D$，$R = \overline{D}$，从结构上消除 S-R 同时有效的未定状态。

$$Q(t+1) = \begin{cases} Q(t), & C = 0 \\ D(t), & C = 1 \end{cases}$$

- $C = 1$：锁存器透明，输出跟随 D。
- $C = 0$：锁存器关闭，保持原状态。
- D 锁存器也可用传输门 (Transmission Gates) 实现。

2.4 锁存器的时序问题 (Latch Timing Problem)

锁存器是**电平敏感**器件。只要控制端处于有效电平，输入到输出路径就可能保持打开。如果输出再经组合逻辑反馈到输入，就可能在一个时钟脉冲内多次变化。

设计目标

时序存储元件在一个时钟脉冲内应只改变一次状态。为打断输入到输出的内部路径，常用两种方案：主从触发器 (Master-Slave Flip-Flop) 和边沿触发器 (Edge-Triggered Flip-Flop)。

2.5 主从触发器 (Master-Slave Flip-Flop)

主从 S-R 触发器由两个带时钟的 S-R 锁存器串联构成，第二级使用反相时钟。

- 主锁存器在 $C = 1$ 时观察输入。

- 从锁存器在 $C = 0$ 时更新输出。
- 通过主从两级使用相反时钟电平，打断从输入到输出的直接通路。
- 输出变化会被推迟到时钟脉冲结束，导致电路速度变慢。

主从 S-R 触发器还存在 **1s catching** 问题：当 $C = 1$ 时，S 或 R 的短暂 1 可能被主锁存器捕获，并在随后传递给从锁存器。

2.6 边沿触发 D 触发器 (Edge-Triggered D Flip-Flop)

边沿触发器只在时钟跳变瞬间采样输入，在时钟保持稳定电平期间忽略输入变化。D 触发器是同步时序设计中最常用的标准存储元件：

$$\boxed{Q(t+1) = D(t)}$$

- **负边沿触发 D 触发器**：输出在时钟下降沿更新。
- **正边沿触发 D 触发器**：输出在时钟上升沿更新，是课件后续同步电路的默认选择。
- D 输入替代 S/R 后，可避免 S-R 主从触发器中的 1s catching 行为。

2.7 直接输入与初始化 (Direct Inputs)

上电或复位时，时序电路通常需要进入已知状态。许多触发器提供异步直接置位/清零输入，绕过时钟立即控制内部锁存器。

R	S	**Clock**	D	**结果**
0	1	X	X	异步 Reset，$Q = 0$
1	0	X	X	异步 Set，$Q = 1$
1	1	↑	0/1	正常边沿触发，$Q^+ = D$
0	0	X	X	状态未定义

2.8 触发器时序参数 (Flip-Flop Timing Parameters)

必须掌握以下参数：

- **建立时间 t_s (Setup Time)**：触发边沿到来之前，输入必须提前稳定的时间。
- **保持时间 t_h (Hold Time)**：触发边沿之后，输入仍需保持稳定的时间，常接近 0。
- **时钟脉宽 t_w (Clock Pulse Width)**：高/低电平持续时间需满足最小要求。
- **传播延迟 t_{pd}**：从触发边沿到输出改变的时间，常用

$$t_{pd} = \max(t_{PHL}, t_{PLH})$$

主从触发器的建立时间常等于触发脉冲宽度，而边沿触发器的建立时间通常远小于脉冲宽度，因此边沿触发器更利于高速设计。

2.9 J-K 与 T 触发器 (J-K and T Flip-Flops)

J-K 触发器类似 S-R 触发器，但允许 $J = K = 1$，此时状态翻转。

J	K	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	$\overline{Q(t)}$

T 触发器只有一个输入 T，等价于令 J-K 触发器 $J = K = T$：

T	$Q(t+1)$
0	$Q(t)$，保持
1	$\overline{Q(t)}$，翻转

补充：J-K 和 T 触发器都可用边沿触发 D 触发器作为核心实现，例如 $D = T \oplus Q$。T 输入本身不能把触发器初始化到指定状态，因此 Reset 通常必不可少。

3 同步时序逻辑分析 (Synchronous Sequential Logic Analysis)

3.1 分析任务与一般流程

时序电路分析的目标是：给定逻辑图，推导状态表、状态图和外部行为。

1. 写出输出方程、触发器输入方程和下一状态方程。
2. 建立带状态的真值表：输入为外部输入和当前状态，输出为电路输出和下一状态。
3. 列出所有状态。
4. 画出状态图。
5. 分析外部功能。
6. 验证正确性，检查自恢复能力，并分析时序约束。

3.2 状态表与状态图 (State Table and State Diagram)

状态表通常包含四部分：

- **Present State**：当前状态变量取值。
- **Input**：外部输入组合。
- **Next State**：由当前状态和输入决定的下一状态。
- **Output**：由当前状态以及有时由输入共同决定的输出。

状态图由状态圆圈、有向边和标注构成。Mealy 型输出通常标在边上，格式为 “输入/输出”；Moore 型输出通常标在状态中。

3.3 分析例 1：两个 D 触发器电路

课件例 1 的状态变量为 $A(t), B(t)$，输入为 $x(t)$，输出为 $y(t)$。由逻辑图可得：

$$D_A = x(A + B), \qquad D_B = x\overline{A}$$

$$A(t+1) = x(A + B), \qquad B(t+1) = x\overline{A}$$

$$y(t) = \overline{x}(A + B)$$

A	B	x	A^+	B^+	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

因为输出 y 与输入 x 直接相关，所以这是 **Mealy 型**电路。

3.4 Moore、Mealy 与混合输出

- **Moore Model**：输出只由状态决定，状态图中常写作 S/y。
- **Mealy Model**：输出由状态和输入共同决定，状态图中常写作 x/y。
- **混合模型**：实际设计中部分输出可为 Moore，部分输出可为 Mealy，以简化输出描述。

Moore 与 Mealy 的复习比较

- Mealy 输出可在输入变化后更快响应，通常状态数较少。
- Moore 输出只随状态变化，行为更稳定，通常需要更多状态。
- 序列检测器中常见记忆口诀：**Moore is More**。

3.5 等价状态 (Equivalent States)

两个状态等价，当且仅当对每一种可能的输入序列，它们产生完全相同的输出序列。等价地说：

- 对每个输入符号，两个状态产生的输出相同；
- 对每个输入符号，两个状态的下一状态相同或等价。

等价状态可以合并，合并后可能暴露新的等价状态，因此状态化简通常是迭代过程。

3.6 分析例 2：三触发器无输入电路

课件例 2 中输入为空，状态变量为 A, B, C，输出 $Z = A$，Reset 初始化到 000。由逻辑图得：

$$A^+ = BC, \qquad B^+ = \overline{B}C + B\overline{C}, \qquad C^+ = \overline{A}\,\overline{C}, \qquad Z = A$$

A	B	C	A^+	B^+	C^+	Z
0	0	0	0	0	1	0
0	0	1	0	1	0	0
0	1	0	0	1	1	0
0	1	1	1	0	0	0
1	0	0	0	0	0	1
1	0	1	0	1	0	1
1	1	0	0	1	0	1
1	1	1	1	0	0	1

从 Reset 后的状态序列为：

$$000 \to 001 \to 010 \to 011 \to 100 \to 000$$

因此它产生一个周期为 5 的状态循环，并在状态 100 时令输出 $Z = 1$。未用状态 $101, 110, 111$ 分别进入 $010, 010, 100$，具有自恢复能力。

3.7 系统级时序约束 (Circuit and System Level Timing)

同步系统中，数据从一个触发器输出，经组合逻辑传播到下一个触发器输入。若时钟周期太短，数据无法在建立时间窗口前稳定，系统会出错。

关键约束为：

$$t_p = t_{\text{slack}} + t_{pd,FF} + t_{pd,COMB} + t_s$$

正确工作要求所有触发器到触发器路径满足：

$$\boxed{t_p \geq \max(t_{pd,FF} + t_{pd,COMB} + t_s)}$$

其中 $t_{\text{slack}} \geq 0$。更精确分析可分别使用 t_{PHL} 与 t_{PLH}，并考虑路径反相次数。

课件计算例

设 $t_{pd,FF} = 1.0$ ns，时钟频率 250 MHz，故 $t_p = 4.0$ ns。

- **边沿触发器**：$t_s = 0.3$ ns

$$4.0 \geq 1.0 + t_{pd,COMB} + 0.3 \Rightarrow t_{pd,COMB} \leq 2.7 \text{ ns}$$

若平均每个门延迟 0.3 ns，约可容纳 9 级门。

- **主从触发器**：$t_s = t_{wH} = 2.0$ ns

$$4.0 \geq 1.0 + t_{pd,COMB} + 2.0 \Rightarrow t_{pd,COMB} \leq 1.0 \text{ ns}$$

约只能容纳 3 级门。

4 同步时序逻辑设计 (Synchronous Sequential Logic Design)

4.1 设计流程 (Design Procedure)

同步时序电路设计的标准流程如下：

1. **Specification**：明确电路功能、输入、输出、复位和时序要求。
2. **Formulation**：得到状态图或状态表。
3. **State Assignment**：为抽象状态分配二进制编码。
4. **Flip-Flop Input Equations**：选择触发器类型，并由下一状态表推导触发器输入方程。
5. **Output Equations**：由输出表推导输出方程。
6. **Optimization**：用卡诺图或布尔代数优化方程。
7. **Technology Mapping**：映射到具体触发器和门电路技术。
8. **Verification**：验证功能、时序、自恢复和非法状态行为。

4.2 状态图构造：序列识别器 (Sequence Recognizer)

状态是对过去输入历史的抽象。对于序列识别器，状态用于记住 “目标序列的哪个前缀已经出现”。

构造步骤：

1. 从初始状态开始，表示还没有识别到目标序列的任何有效前缀。
2. 为目标序列的第一个符号建立状态。
3. 继续为每一个连续前缀建立状态。
4. 最终状态或最终转移表示完整序列被识别。
5. 补全所有非目标输入的转移。
6. 特别处理**重叠序列**，保证目标序列无论从输入流的哪个位置开始都能被识别。

4.3 例：识别序列 1101 的 Mealy 模型

设输出 1 表示刚刚识别到序列 1101。Mealy 模型状态含义：

- A：未出现任何有效前缀。
- B：已经出现前缀 1。
- C：已经出现前缀 11，或连续两个以上的 1。
- D：已经出现前缀 110。

Present State	**Next State**		**Output**	
	$x=0$	$x=1$	$x=0$	$x=1$
A	A	B	0	0
B	A	C	0	0
C	D	C	0	0
D	A	B	0	1

从 D 状态遇到 1 时识别出 1101，但最后这个 1 又可作为新一次识别的前缀，因此转到 B，支持重叠识别，例如 1101101 中的两次出现。

4.4 例：识别 1101 的 Moore 模型

Moore 模型输出必须绑定在状态上，因此需要额外状态 E 表示 “完整序列已经出现，当前输出为 1”。

Present State	Next State		Output y
	$x=0$	$x=1$	
A	A	B	0
B	A	C	0
C	D	C	0
D	A	E	0
E	A	C	1

状态 E 和 B 对未来输入的响应相似，但当前输出不同，因此不能合并。

4.5 状态编码 (State Assignment)

若有 m 个状态，最少需要 n 位编码：

$$n = \lceil \log_2 m \rceil$$

使用 n 位编码时有 $2^n - m$ 个未用状态。不同编码会改变输出方程、激励方程和电路复杂度。

状态编码四条经验规则

1. 对相同输入具有相同下一状态的当前状态，尽量分配相邻编码。
2. 同一当前状态在相邻输入下得到的下一状态，尽量分配相邻编码。
3. 具有相同输出的状态，尽量分配相邻编码。
4. 初始状态或最常用状态优先分配为 0。

优先级上，第一条通常最重要；若状态对在多条规则中多次出现，应优先让这些状态相邻。

4.6 状态编码例：Counting vs. Gray

对于 1101 的 Mealy 状态表，若采用 Counting Order：

$$A = 00, \quad B = 01, \quad C = 10, \quad D = 11$$

设状态位为 Y_1Y_2，输入为 X，使用 D 触发器时可得一组方程：

$$D_1 = Y_1\overline{Y_2} + X\overline{Y_1}Y_2$$

$$D_2 = \overline{X}Y_1\overline{Y_2} + X\overline{Y_1Y_2} + XY_1Y_2$$

$$Z = XY_1Y_2$$

若采用 Gray Code：

$$A = 00, \quad B = 01, \quad C = 11, \quad D = 10$$

可得到更简单的一组方程：

$$D_1 = Y_1Y_2 + XY_2, \qquad D_2 = X, \qquad Z = XY_1\overline{Y_2}$$

这说明状态编码会显著影响电路成本。优化后还需进行技术映射，例如映射到带 Reset 的 D 触发器、NAND 门和反相器。

4.7 状态编码规则例

课件给出状态转移/输出表：

Present State	$X = 0$	$X = 1$
A	$C/0$	$D/0$
B	$C/0$	$A/0$
C	$B/0$	$D/0$
D	$A/1$	$B/1$

按规则分析：

- 规则 1 倾向：$A-B$、$A-C$ 相邻。
- 规则 2 倾向：$C-D$、$A-C$、$B-D$、$A-B$ 相邻。
- 规则 3 倾向：$A-B$、$A-C$、$B-C$ 相邻。
- 规则 4：令 $A = 00$。

一种合适编码为：

$$A = 00, \quad B = 01, \quad C = 10, \quad D = 11$$

对应编码后的表为：

Present State	$X = 0$	$X = 1$
00 (A)	10/0	11/0
01 (B)	10/0	00/0
10 (C)	01/0	11/0
11 (D)	00/1	01/1

4.8 设计例 3：模 3 累加器

目标：设计一个 2 位操作数的模 3 累加器。

- **Modulo n Adder**：输出为加法结果除以 n 的余数。例如 $2+2 \pmod 3 = 1$。
- **Accumulator**：随时间累计输入操作数，并把累计和保存在状态中。
- 状态/输出：$(Y_1, Y_0) = (Z_1, Z_0)$，表示当前模 3 和。
- 有效输入：$00, 01, 10$。输入 11 对模 3 操作数而言作为 don't care。

Present State	**Next State for X_1X_0**				**Output**
Y_1Y_0	00	01	11	10	Z_1Z_0
00 (A)	00	01	X	10	00
01 (B)	01	10	X	00	01
11 (unused)	X	X	X	X	11
10 (C)	10	00	X	01	10

使用 D 触发器，并把无效输入和未用状态作为 don't care 优化，可得到一组可用方程：

$$D_1 = \overline{Y_1}\,\overline{Y_0}X_1 + Y_0X_0 + Y_1\overline{X_1}\,\overline{X_0}$$

$$D_0 = \overline{Y_1}\,\overline{Y_0}X_0 + Y_0\overline{X_1}\,\overline{X_0} + Y_1X_1$$

$$Z_1 = Y_1, \qquad Z_0 = Y_0$$

4.9 未用状态与自启动 (Unused States and Self-Starting)

状态编码可能产生未用状态。电路上电、噪声或干扰可能使 FSM 进入这些状态。

- 若未用状态指向自己或互相循环，电路可能“冻结”在非法区域。
- 若所有未用状态都能回到合法状态，称为 **self-starting**。
- 可通过 Reset 逻辑强制进入初始状态。
- 若非法状态不可接受，可设计 ERROR 状态并输出错误码。
- 若使用 don't care 优化，应在最终硬件实现后检查非法状态行为，而不能只看抽象状态表。

5 经典时序逻辑电路 (Classic Sequential Logic Circuits)

5.1 寄存器 (Registers)

寄存器是一组二进制存储元件的集合，可保存多位数据。它既可被看作时序逻辑电路，也可被看作存储一个二进制向量的硬件模块。

- 一个触发器只能存 1 位；n 位寄存器由 n 个触发器组成。
- n 位寄存器有 2^n 个状态、2^n 种输入组合和 2^n 种输出组合。
- 简单并行寄存器中，下一状态等于输入向量，输出等于当前状态向量。
- 寄存器是现代处理器中的核心部件，作为比主存更快的临时存储。

2 位寄存器示例

设输入为 In_1In_0，状态为 A_1A_0，输出为 Y_1Y_0。则：

$$A_1(t+1)A_0(t+1) = In_1In_0, \qquad Y_1Y_0 = A_1A_0$$

这是 Moore 型，因为输出只由寄存器当前状态决定。

5.2 使能与时钟门控 (Enable and Clock Gating)

课件展示了 4 位寄存器的 Clock Gating 以及带 EN 的触发器。复习时可抓住两种实现思想：

- **Clock Gating**：关闭某些寄存器的时钟以节能，但必须避免门控时钟产生毛刺，实际芯片中通常使用专门的时钟门控单元。
- **Flip-Flop with EN**：用使能控制是否加载新值，常见逻辑为

$$D_{\text{in}} = \begin{cases} D_{\text{new}}, & EN = 1 \\ Q, & EN = 0 \end{cases}$$

即用多路选择器在新输入和保持值之间选择。

5.3 寄存器传送与微操作 (Register Transfers and Microoperations)

寄存器传送操作描述寄存器中数据的移动和处理，由三部分指定：

- 系统中的寄存器集合。
- 对寄存器数据执行的操作。
- 控制这些操作发生顺序的控制信号。

微操作是寄存器数据上的基本操作，例如加载、计数、加法、减法、移位。常见四类：

- **Transfer**：寄存器间移动数据。
- **Arithmetic**：加、减、乘、除等算术操作。
- **Logic**：按位 OR、AND、XOR、NOT 等逻辑操作。
- **Shift**：寄存器内数据左右移动。

控制表达式

控制表达式写在操作左侧，用冒号分隔：

$$P : R_2 \leftarrow R_1$$

当 $P = 1$ 时操作发生，$P = 0$ 时操作不发生。

加减法示例：

$$\overline{X}K_1 : R_1 \leftarrow R_1 + R_2$$

$$XK_1 : R_1 \leftarrow R_1 + \overline{R_2} + 1$$

当 $X = 0$ 时执行加法；当 $X = 1$ 时加上 R_2 的补码，实现减法。

5.4 寄存器传送结构 (Register Transfer Structures)

常见结构包括：

- **专用多路选择器 (Dedicated Multiplexers)**：每个寄存器前有自己的 MUX，常用于移位寄存器和计数器。
- **总线型传送 (Bus-Based Transfers)**：多个输入经共享 MUX 驱动总线，再送到多个寄存器。
- **三态总线 (Three-State Bus)**：多个三态驱动器连接到同一总线，同一时刻只允许一个驱动器有效。
- **混合结构**：多个 MUX、多个总线和三态结构组合使用。

5.5 移位寄存器 (Shift Registers)

移位寄存器能把保存的位向 MSB 或 LSB 方向移动。最简单形式是把一排 D 触发器串联：

$$In \rightarrow A \rightarrow B \rightarrow C \rightarrow Out$$

- In：串行输入，也称 shift right input。
- Out：串行输出。
- (A, B, C, Out)：若可同时访问，则称为并行输出。

串行移位行为

初始未知用 ? 表示，输入序列对应课件表格：

时刻	In	A	B	C	Out
T_0	0	?	?	?	?
T_1	1	0	?	?	?
T_2	1	1	0	?	?
T_3	0	1	1	0	?
T_4	1	0	1	1	0
T_5	1	1	0	1	1
T_6	1	1	1	0	1

并行加载移位寄存器

在每一级 D 触发器前加入多路选择器，可在“移位”和“并行加载”之间选择。

Shift	Load	Operation
0	0	No Change
0	1	Parallel Load
1	X	Shift Down

若每级前使用 4 输入 MUX，还可统一支持保持、下移、上移、并行加载：

S_1	S_0	Register Operation
0	0	No change
0	1	Shift down
1	0	Shift up
1	1	Parallel load

5.6 计数器 (Counters)

计数器是按特定状态序列循环的时序电路。它可以向上计数、向下计数，或按任意固定序列计数。其状态值本身通常就是输出。

- 可用作简单时钟或事件计数器。
- 可记录发送/接收了多少位、计算进行了多少步。
- 所有处理器都有程序计数器 PC，用于记录当前执行指令位置，通常随时钟递增。

5.7 波纹计数器 (Ripple Counters)

波纹计数器中，时钟只直接连接到最低位触发器；更高位触发器由前一级输出触发。因此它并非真正同步。

- 最低位每个时钟边沿翻转。
- 每个更高位的翻转频率是前一位的一半。
- 状态变化从 LSB 向 MSB 逐级传播，称为 ripple。
- n 位最坏传播延迟约为 $n \cdot t_{PHL}$ 或对应方向传播延迟之和。
- 优点是结构简单、功耗较低；缺点是高位变化滞后，易产生中间暂态。

3 位向上计数示例：

$$000 \to 001 \to 010 \to 011 \to 100 \to 101 \to 110 \to 111 \to 000$$

5.8 同步计数器 (Synchronous Counters)

同步计数器让所有触发器共享同一时钟，通过组合逻辑生成下一状态。对于二进制向上计数器，可使用增量器：

- XOR 用于决定某一位是否翻转。
- 某位翻转的条件是所有低位均为 1。
- Count Enable 可强制保持状态。
- Carry Out 可连接到更高位计数器的 Count Enable，实现级联。

若用串联 AND 形成进位链，会产生较长路径延迟，称为 **serial gating**。可改用并行 AND 或类似超前进位思想实现 **parallel gating**，减少大规模计数器的长路径。

5.9 模 N 计数器与 BCD 计数器

模 N 计数器按 N 个状态循环，计数值相当于除以 N 的余数。N 不一定是 2 的幂；模 10 计数器即 BCD 计数器。

同步 BCD 计数器：D 触发器实现

BCD 计数器合法状态为 0000 到 1001，其中 $Q_8Q_4Q_2Q_1$ 表示十进制 0 到 9；状态 1010 到 1111 可作为 don't care 优化。常见优化方程为：

$$D_1 = \overline{Q_1}$$

$$D_2 = \overline{Q_8}\,\overline{Q_2}Q_1 + Q_2\overline{Q_1}$$

$$D_4 = \overline{Q_4}Q_2Q_1 + Q_4\overline{Q_2} + Q_4\overline{Q_1}$$

$$D_8 = Q_8\overline{Q_1} + Q_4Q_2Q_1$$

终端计数/进位输出常取：

$$Y = Q_8Q_1$$

非法状态行为

由上述方程可得 6 个非法状态的实际下一状态：

Present State	**Decimal**	**Next State**
1010	10	1011
1011	11	0110
1100	12	1101
1101	13	0100
1110	14	1111
1111	15	0010

该设计若进入非法状态，会在两个时钟周期内回到合法状态。是否足够取决于应用和设计规范。可选措施包括：

- 增加非法状态检测信号，例如

$$Invalid = Q_8(Q_4 + Q_2)$$

- 修改逻辑使非法状态一个周期内回到合法状态。
- 强制非法状态回到指定状态，如 0000。

同步 BCD 计数器：T 触发器实现

若用 T 触发器，利用 $T = Q \oplus Q^+$，可得到一组常用激励方程：

$$T_1 = 1, \qquad T_2 = \overline{Q_8}Q_1, \qquad T_4 = Q_2Q_1, \qquad T_8 = Q_1(Q_8 + Q_4Q_2)$$

同样需要处理 1010 到 1111 的 don't care 与自恢复问题。

5.10 模 N 计数的实现方法

课件总结了用 n 位二进制计数器构造模 N 计数器的几类方法：

- 检测终端计数 N，异步 Clear 到 0 或异步 Load 0。该方法会让终端状态只存在极短时间，可能因时序条件失败。
- 检测终端计数 $N-1$，同步 Clear 到 0。
- 检测终端计数 $N-1$，同步 Load 0。
- 检测某个终端计数，并用 Load 预置为“终端计数值减去 $N-1$”。
- 完全按状态表自定义设计模 N 计数器，例如 BCD 计数器。

模 7：检测 7 后异步清零

用带异步 Clear 的 4 位同步二进制计数器检测 7 并清零，会得到：

$$0, 1, 2, 3, 4, 5, 6, 7(\text{short}), 0, \ldots$$

状态 7 只短暂出现，课件称为 **suicide counter**。这种设计可能因门延迟和异步清零竞争而失败，复习时记住：**不要这样做**。

模 7：检测 6 后同步加载 0

更可靠方法是检测终端合法状态 6，在下一个时钟同步 Load 0：

$$0, 1, 2, 3, 4, 5, 6, 0, 1, \ldots$$

若把 7 以上状态作为 don't care，检测 6 可简化为：

$$Load = Q_2 Q_1$$

模 6：预置 9，检测 14 后同步加载 9

用 4 位同步二进制计数器实现模 6 的另一种思路是让它循环：

$$9, 10, 11, 12, 13, 14, 9, \ldots$$

Reset 时预置 9，检测到 14 后同步 Load 9。若终端计数为 15，许多计数器内部已有 Carry Out 可用。

60 进制分钟计数

分钟的十进制显示可用两个 BCD/模计数器级联：

- 个位：模 10，计数 0 到 9。
- 十位：模 6，计数 0 到 5。
- 个位从 9 回到 0 时给十位一个进位。
- 整体从 59 回到 00，实现模 60。

6 全章总结 (Summary)

本章围绕“数字系统如何记住过去”展开。复习时建议抓住以下主线：

1. **状态来自反馈与存储**：组合逻辑一旦引入反馈和延迟，就可能保存过去的值；稳定存储需要锁存器和触发器。
2. **锁存器是电平敏感，触发器是边沿敏感**：D 锁存器透明时会持续跟随输入；边沿触发 D 触发器只在时钟边沿采样，公式 $Q^+ = D$ 是同步设计核心。
3. **S-R、J-K、T、D 各有状态表**：S-R 有禁止输入；J-K 的 11 翻转；T 的 1 翻转；D 直接保存输入。
4. **同步分析从方程到状态图**：先写触发器输入方程、下一状态方程和输出方程，再建状态表、状态图，最后分析功能与自恢复。
5. **同步设计从规格到硬件**：规格说明、状态图、状态编码、触发器方程、输出方程、优化、技术映射和验证缺一不可。
6. **状态编码影响成本**：相邻编码可简化卡诺图；Gray 编码、初始状态取 0、相同输出相邻等规则都是工程经验。
7. **时序约束决定最高频率**：所有触发器到触发器路径必须满足

$$t_p \geq t_{pd,FF} + t_{pd,COMB} + t_s$$

8. **寄存器、移位寄存器、计数器是常用积木**：寄存器保存向量，移位寄存器移动位，计数器按固定状态序列循环。
9. **异步清零构造模计数器风险高**：检测 N 后异步清零会产生短暂非法脉冲；更推荐检测 $N-1$ 后同步清零或同步加载。
10. **不要忽视非法状态**：don't care 可降低成本，但最终必须检查非法状态是否能自恢复，必要时加入 Reset、Invalid 或 ERROR 逻辑。

“组合逻辑回答现在该做什么；时序逻辑记住过去，从而决定下一步。”